

PROJECT PROPOSAL FOR HACKENZA 2026

**The Robust Vision Challenge:
Adaptation Under Distributional Shift & Label Noise**

Model Title:
**An Autonomous Pipeline for Robust Vision
Adaptation**

Team Kingsmen

Aayush Kushwaha (2024A7PS0524G)
Anshul D Jain (2024ADPS0844G)
Vaiebhav Shreevarshan R (2024AAPS1427G)
Divyam Agarwal (2024A7PS1442G)

1 Abstract

The transition of computer vision models from curated datasets to autonomous deployment introduces a “triad of failure” that standard machine learning pipelines are ill-equipped to handle: **Label Noise**, **Covariate Shift**, and **Label Shift**. In high-stakes environments such as autonomous manufacturing and edge surveillance, these three phenomena occur simultaneously, creating a significant gap between model training and real-world performance.

The problem demands a system capable of achieving high accuracy on a clean validation set despite training on “toxic” data, while simultaneously adapting to unknown sensor noise and estimating target class distributions on the fly. This must be accomplished under strict constraints: the model must be initialized from scratch, and no external clean data or manual intervention is permitted.

Consequently, the challenge is not merely one of classification, but of engineering a self-correcting pipeline that aligns disparate distributions in real-time.

2 Problem Understanding and Constraints

Our analysis of the challenge confirms that it mimics the messy reality of production-grade AI. We are building a system that can withstand corrupted inputs and environment shifts.

2.1 Technical Interpretation of the Challenge

We have identified three distinct failure points that our pipeline must navigate:

- **Data Poisoning (The Training Barrier):** We recognize that 30% of the labels in `source_toxic.pt` are incorrect. Our understanding is that a standard training approach will treat this noise as “ground truth”, leading to a model that learns patterns of error rather than actual features.
- **Sensor Degradation (The Input Barrier):** The introduction of **Impulse Noise** in the target stream represents a shift in data distribution. We must ensure our model remains functional when the “image quality” drops significantly from what it saw during training.
- **Population Drift (The Logic Barrier):** Because class ratios in the target environment are unknown and skewed, we cannot rely on the balanced distribution of the training set. We must actively estimate the target class weights on the fly to avoid biased predictions.

2.2 Commitments to Constraints

To maintain the integrity of the challenge, we are strictly adhering to the “Hard Mode” rules:

- **No Pre-trained Weights:** We are starting with a blank slate. Our model will learn its feature extraction capabilities entirely from the provided noisy data.

- **No Training-Time Augmentation:** We will not use synthetic data augmentation during the training phase. The model must achieve robustness through its architecture and loss logic and not through sheer volume of transformed images.
- **No External Data:** We will be using only the provided `.pt` files.

3 Proposed Architecture & Methodology

3.1 The Baseline: Custom ResNet-9

We have selected a **custom ResNet-9 based architecture**, built from scratch, as our foundation. We believe this is the optimal choice for Fashion-MNIST because it provides enough depth to learn complex features while being lean enough to resist the memorization of noisy labels.

3.2 Phase 1: Robust Training Pipeline

The goal is to “decontaminate” the training process by using a loss function that remains robust despite a **30% symmetric label flip** in the `source_toxic.pt` dataset. Instead of using a single loss, we implement an Active–Passive Loss (APL) framework.

The Problem with Standard Loss Functions

- **The Weighting Issue:** Standard Categorical Cross-Entropy (CCE) is highly biased toward labels, even if they are wrong, due to its exponential penalty.
- **Noise Memorization:** In small sample sizes, any label noise generates massive penalties under **standard Cross-Entropy**. This excessive gradient pressure overrides pattern recognition, *forcing the neural network to memorize erroneous label associations*.
- **The Inadequacy of Robustness Alone:** Purely robust losses like Mean Absolute Error (MAE) are too “passive”; they lack the gradient signal strength to effectively train deep architectures on complex fashion textures, resulting in **underfitting**. Hence the model stalls.

The Proposed Solution: Active–Passive Loss (APL)

- **Active Loss:** We utilize Generalized Cross Entropy (GCE) as the active component. It provides the necessary gradient scaling to drive rapid convergence on the structural features of the clothing items.
- **Passive Loss:** We pair this with a **Normalized Loss** (such as Normalized Cross Entropy) that acts as the passive component. This component constrains the global risk and balances the active gradients, ensuring that the model cannot be dragged toward the poisoned labels.

Why APL is the Best Solution for Noisy Fashion-MNIST?

- By integrating GCE within the APL framework, we achieve a critical balance; it retains the **convergence speed** of CCE for identifying complex textures while adopting the **noise immunity** of MAE.
- This APL framework takes advantage of the early learning phenomena of the neural network.
- The active component pushes the model to learn the patterns, while the passive component limits the influence of the poisoned labels.

3.2.1 Architectural Justification: Why ResNet-9?

We selected a **Group-Normalized ResNet-9** over deeper models (ResNet-50) or Transformers for three strategic reasons:

- **Optimal Capacity:** Deep models (ResNet-50/101) possess excess parameters that facilitate the “memorization” of the 30% label noise. ResNet-9 provides a “Goldilocks” capacity that is deep enough to learn complex features and shallow enough to reject noise.
- **Immunity to Batch Shift:** Standard models use Batch Normalization, which fails during Test-Time Adaptation if the target stream is skewed. By using **Group Normalization (GN)**, we decouple normalization from batch statistics, enabling single-sample inference at test time.
- **Adaptation Speed:** Test-Time Adaptation requires real-time gradient updates. ResNet-9 offers the low-latency overhead necessary for the **Sharpness-Aware Minimization (SAM)** updates without the massive computational overhead of deeper architectures.

3.2.2 Summary of Alternatives

- **vs. VGG:** VGG lacks **Skip Connections**, which are required to create the “flat loss landscape” that our Phase 3 adaptation relies on.
- **vs. Transformers:** ViTs require massive clean data to converge; they lack the **Inductive Bias** (spatial locality) that our ResNet-9 uses to navigate noisy Fashion-MNIST images.
- **vs. ResNet-50:** On 28×28 images, the extra depth increases the risk of memorizing “toxic” label patterns without improving feature extraction.

Table 1: ResNet-9 Architecture Summary

Layer	Type	Configuration	Output Shape
Input	Image	Grayscale	$1 \times 28 \times 28$
Stem	Conv2d	3×3 , Padding 1	$64 \times 28 \times 28$
Downsample	MaxPool	2×2 , Stride 2	$64 \times 14 \times 14$
ResBlock 1	Residual	[Conv(3×3) +GN + ReLU] $\times 2$	$128 \times 14 \times 14$
Transition	Conv2d	3×3 , Stride 2	$256 \times 7 \times 7$
ResBlock 2	Residual	[Conv(3×3) +GN + ReLU] $\times 2$	$256 \times 7 \times 7$
Neck	Global Pool	Global Average Pooling	$256 \times 1 \times 1$
Head	Fully Conn.	Linear + Softmax	10 (Classes)

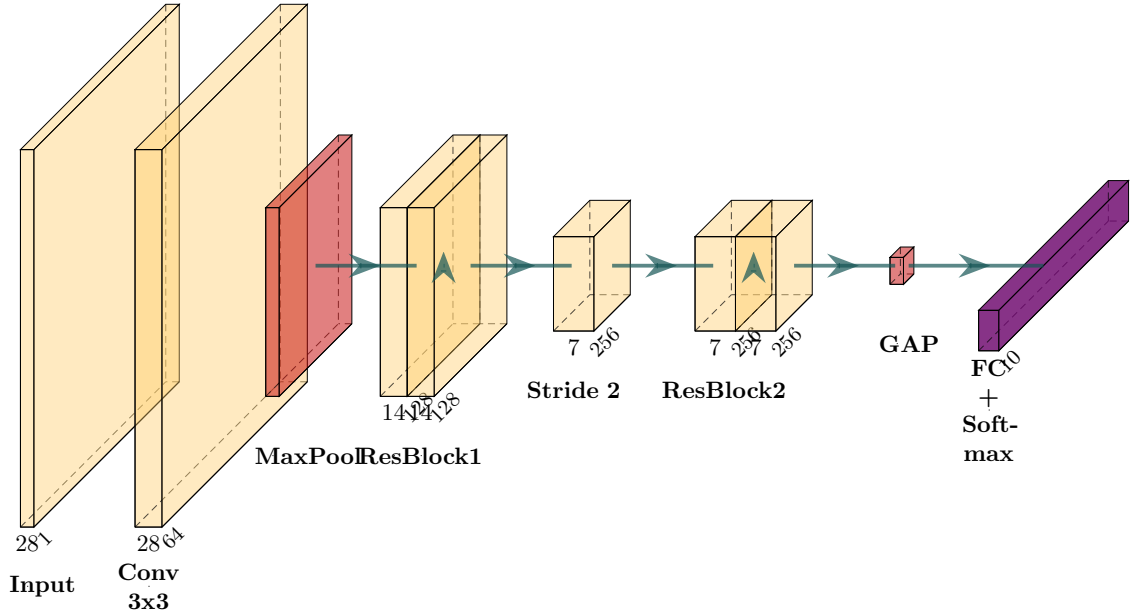


Figure 1: Project Architecture Overview

3.3 Phase 2: Distribution Estimation (Reconnaissance)

Upon successfully decontaminating the training process, the pipeline addresses Population Drift (Label Shift), where the class distribution of the target environment (`target_static.pt`) diverges sharply from the source training data (`source_toxic.pt`).

The engineering protocol demands the implementation of a Label Shift Estimator to calculate an “estimated weights” vector ($\hat{w}_t$). The crucial asset for this operation is the `val_sanity.pt` dataset. Because this subset is entirely clean, it provides the exact theoretical mechanism required to calibrate a shift estimator without relying on the poisoned source data.

While Maximum Likelihood Label Shift (MLLS) and Expectation-Maximization (EM) are common methodologies for deriving these estimated weights, MLLS utilizes an iterative

algorithm that requires the base classifier to be highly calibrated. DNNs trained on poisoned data are uncalibrated, which would cause the MLLS likelihood estimations to produce highly erratic weight vectors.

Instead, the pipeline implements **Black Box Shift Estimation (BBSE)**. BBSE bypasses the need for perfectly calibrated confidences by leveraging the law of total probability and linear algebra to directly solve for the target distribution. The estimation is executed in three steps:

1. **Confusion Matrix Construction:** The GCE-trained ResNet-9 generates predictions over clean images in the `val_sanity.pt` dataset. It enables the exact empirical construction of the confusion matrix $\hat{C}_{\hat{y},y}$. To guarantee matrix invertibility the pipeline utilizes **BBSE-Soft**, aggregating continuous softmax probabilities.
2. **Target Marginal Calculation:** The frozen ResNet-9 is subsequently executed over the entirely unlabelled `target_static.pt` data stream. The continuous softmax outputs of the network are averaged across the dataset to form the empirical target mean prediction vector $\hat{\mu}_{\hat{y}}$.
3. **Matrix Inversion and Weight Extraction:** The final vector of estimated weights $\hat{w}_t$ is algorithmically recovered by inverting the soft confusion matrix and computing the dot product with the target prediction means:

$$\hat{w}_t = \hat{C}_{\hat{y},y}^{-1} \hat{\mu}_{\hat{y}} \quad (1)$$

3.4 Phase 3: Test-Time Adaptation (Alignment)

Upon completing the robust training and label shift estimation stages, the pipeline combats continuous Sensor Degradation (Covariate Shift).

By integrating the class weights ($\hat{w}_t$) estimated in Phase 2, this stage ensures the model remains performant under unknown corruption. Before parameter updates are performed, model predictions are corrected using these weights so that online learning respects the detected population shift. The goal is to maximize target-stream accuracy through autonomous, real-time parameter adjustment to stay reliable in changing conditions of distributional shift.

To achieve this, the pipeline implements **Sharpness-Aware and Reliable (SAR)** entropy minimization. The adaptation is executed in three steps:

1. **Label-Shift-Aware Posterior Correction:** Before any parameter updates, the model’s raw predictions are first adjusted using the class weight vector ($\hat{w}_t$) estimated through BBSE in Phase 2. Utilizing Bayes’ theorem principles, this probability scaling ensures that the model’s updates remain strictly aligned with the detected population drift. This ensures that the unsupervised adaptation process is grounded in the actual class distribution.
2. **Reliable Sample Selection (Entropy Filtering):** SAR evaluates the Shannon entropy of the corrected predictions on incoming samples. By filtering out these unrecoverable, noise-induced inputs, the pipeline ensures that gradient updates are driven by structured visual information, preserving the decontaminated features learned in Phase 1.

3. **Sharpness-Aware Minimization (SAM):** For the remaining confident predictions, the model updates its Group Normalization affine parameters and classifier weights. The adaptation mechanism utilizes Sharpness-Aware Minimization (SAM) to update parameters in a manner that favors mathematically stable and flat regions of the loss landscape. Flat solutions are inherently less sensitive to small perturbations in the input space. This specific mechanism guarantees that the model generalizes seamlessly to novel, unseen corruption types present in the `hidden_eval.pt` dataset.

4 Execution Plan & Timeline

Table 2: Execution Plan & Timeline

Timeline	Phase	Objective & Key Engineering Tasks
Hours 0–4	Phase 0 & 1 Setup: Infrastructure & Architecture	Tasks: Initialize the repository. Build custom PyTorch Dataset loaders for the provided <code>.pt</code> tensor files. Implement the full ResNet-9 with Group Normalization. Code the APL (GCE + Normalized Loss) module.
Hours 4–8	Phase 1 Execution: Decontamination	Tasks: Execute the training loop on <code>source_toxic.pt</code> . Monitor loss convergence to validate GCE’s noise-robust behavior. Validate on <code>val_sanity.pt</code> to confirm that the model’s accuracy is not being degraded by the 30% label noise.
Hours 8–12	Phase 2: Distribution Reconnaissance	Tasks: Freeze the network parameters. Execute inference over <code>val_sanity.pt</code> to construct the soft confusion matrix. Run inference over <code>target_static.pt</code> to calculate the target marginal. Perform the matrix inversion to extract the estimated weight vector $\hat{w}_t$.
Hours 12–18	Phase 3: Test-Time Adaptation (The Bottleneck)	Tasks: Unfreeze the network. Implement the Reliable Sample Selection (Shannon entropy filter). Integrate SAM optimizer. Implement label-shift-aware posterior correction using the $\hat{w}_t$ vector.
Hours 18–22	Synthesis & Forensics	Tasks: Stitch all components into a single, automated <code>train.py</code> script. Format the analytical forensics report.
Hours 22–24	Compliance Audit & Buffer	Tasks: Conduct a rigid code review. Confirm absolute zero external downloads, verify reproducibility, and prepare the final submission.

5 Addressing the Evaluation Criteria

- **Generalization via Sharpness-Aware Minimization:** To succeed on the unseen corruptions in `hidden_eval.pt`, we utilize **Sharpness-Aware and Reliable (SAR) entropy minimization** in Phase 3 to optimize for “flat minima,” ensuring the model is desensitized to novel covariate shifts without using forbidden augmentations.
- **Compliance through Robust Decontamination:** We ensure full reproducibility and compliance by initializing our **ResNet-9** from scratch and employing APL **BASED Generalized Cross Entropy (GCE)** as active part and Normalized Loss as passive part to decontaminate the 30% label noise in `source_toxic.pt`, strictly avoiding pre-trained weights or external datasets.
- **Analytical Forensics via Label Shift Estimation:** The pipeline satisfies the forensics requirement by implementing **Black Box Shift Estimation (BBSE-Soft)** in Phase 2 to automatically detect and quantify target population imbalances ($\hat{\mathbf{w}}_t$) from the unlabeled `target_static.pt` stream.

6 Tech Stack & Implementation Details

Core Frameworks & Ecosystem

- Language: Python 3.10+
- Primary Deep Learning Framework: PyTorch. Given the datasets are provided as `.pt` tensor files, PyTorch is the native and most efficient environment for this pipeline.
- Numerical Computation: `torch.linalg` and NumPy for executing the Black Box Shift Estimation (BBSE) matrix inversions during Phase 2.
- Data Handling: Custom PyTorch `Dataset` and `DataLoader` classes specifically engineered to ingest the raw tensor streams without relying on high-level APIs that might inadvertently trigger default augmentations.

Hardware & Compute Strategy

- Cloud Compute Environment: The end-to-end pipeline will be executed on a Kaggle cloud environment provisioned with dual NVIDIA T4 GPUs.
- Local Prototyping: Initial codebase structuring, custom `.pt` data loader validation, and BBSE matrix inversion logic will be prototyped locally on CPU before full deployment to the Kaggle GPU environment.

References

- [1] H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms,” *arXiv preprint arXiv:1708.07747*, 2017.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [3] Z. Zhang and M. Sabuncu, “Generalized Cross Entropy Loss for Training Deep Neural Networks with Noisy Labels,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.
- [4] A. Ghosh, H. Kumar, and P. Sastry, “Robust Loss Functions under Label Noise for Deep Neural Networks,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 31, no. 1, 2017.
- [5] Z. Lipton, Y.-X. Wang, and A. Smola, “Detecting and Correcting for Label Shift with Black Box Predictors,” in *International Conference on Machine Learning (ICML)*, 2018, pp. 3122–3130.
- [6] Z. Nado, S. Padhy, D. Sculley, A. D’Amour, B. Lakshminarayanan, and J. Snoek, “Evaluating Prediction-Time Batch Normalization for Robustness under Covariate Shift,” *arXiv preprint arXiv:2006.10963*, 2020.
- [7] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, “Tent: Fully Test-Time Adaptation by Entropy Minimization,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [8] Q. Wang, O. Fink, L. Van Gool, and D. Dai, “Continual Test-Time Domain Adaptation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 7201–7211.
- [9] S. Niu, J. Wu, Y. Zhang, Y. Chen, S. Zheng, P. Zhao, and M. Tan, “Towards Stable Test-Time Adaptation in Dynamic Wild World,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [10] X. Ma, H. Huang, Y. Wang, S. Romano, S. Erfani, and J. Bailey, “Normalized Loss Functions: Unifying Mutual Information and Cross Entropy for Deep Learning with Noisy Labels,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2020.